FLUJOGRAMA 03 · FLUJO UNIFICADO END-TO-END

# Flujo Unificado: GLPI + Sistema Propio

Diagrama de secuencia con 4 carriles (Cliente, GLPI, Técnico, Sistema Propio) y detalle explícito de operaciones en BD.

**Versión:** 1.0 **Fecha:** 2026-10-14 **Carriles:** 4 actores / componentes

**Bases de Datos:** MariaDB (GLPI) + PostgreSQL (Auditoría/Preferencias)

1 Service Desk → GLPI

**Service Desk crea la cuenta del usuario** en la interfaz administrativa de GLPI. El service desk *solo crea cuentas, no tickets.*

`INSERT glpi_users (is_active=1)`

**2** Cliente → GLPI

El cliente autenticado entra al portal de autoservicio GLPI y **crea su propio ticket** con su categoría ITIL y canal.

`Portal /helpdesk.public.php`

**3** GLPI → MariaDB

Persistencia del ticket en estado **1 (Nuevo)** con categoría ITIL, canal de entrada y prioridad.

```
INSERT glpi_tickets (status=1)
```

4 GLPI → MariaDB

Vinculación del ticket al grupo resolutor correspondiente según regla de negocio o categoría.

```
INSERT glpi_groups_tickets (type=2)
```

5 Técnico → Sistema

El técnico ingresa a su panel de trabajo y navega a la vista de cola: **/queue**.

```
HTTP GET /queue (React 19)
```

6 Sistema → MariaDB

La API .NET 10 consulta con Dapper los tickets pendientes del grupo técnico que aún no tienen asignado un resolutor.

`SELECT glpi_tickets + groups`

7 Sistema → Técnico

La interfaz muestra la tabla optimizada con filtros rápidos, categoría ITIL y botón destacado de autotoma.

`Render React 19 UI`

8 Técnico → Sistema

El técnico hace clic en **"Asignarme"** sobre el ticket seleccionado.

`POST /api/tickets/{id}/assign-me`

9 Sistema → MariaDB

El backend vincula al usuario logueado como técnico asignado (type=2) y actualiza estado a En Curso (status=2).

`INSERT glpi_tickets_users (type=2)`

**10** Sistema → PostgreSQL

El backend registra el evento en auditoría (PK: `BIGINT GENERATED ALWAYS AS IDENTITY`) para métricas y trazabilidad.

```
INSERT audit_log (action='claim')
```

**11** Técnico → Sistema

El técnico realiza el diagnóstico, redacta la solución técnica en el diálogo y hace clic en **"Resolver Ticket"**.

```
POST /api/tickets/{id}/solution
```

12 Sistema → MariaDB

Se guarda la solución en GLPI y se pasa el ticket a estado **5 (Solucionado)** según ADR-006.

```
INSERT glpi_itilsolutions + UPDATE status=5
```

13 Sistema → PostgreSQL

Se registra el evento de resolución con el tiempo de atención y metadata en el log de auditoría propio.

```
INSERT audit_log (action='resolve')
```

14 Cliente → GLPI

El cliente consulta su portal GLPI o recibe la notificación automática de que su ticket ha sido **Solucionado**.

```
SELECT glpi_tickets (status=5)
```

### Operaciones en MariaDB (GLPI)

Modificaciones controladas estrictamente en tablas nativas de GLPI: asignaciones (`glpi_tickets_users`) y soluciones (`glpi_itilsolutions`). No se tocan tablas estructurales ni configuraciones de GLPI.

### Operaciones en PostgreSQL (Sistema Propio)

Persistencia inmutable de eventos en `audit_log` (claim, resolve) y persistencia de preferencias del técnico. Protegida con transacciones locales sin impacto en el rendimiento de GLPI.